CURRICULUM COMPANION : TECHNICAL LEARNING MODULES & FREE COURSES

Week 2 Learning Materials : Classical ML & Anti-Fraud Pipelines

Leak-Free Ensembles, Class Imbalance, TreeSHAP Explainability & 100% Free Courses

TARGET AUDIENCE	ESTIMATED READING TIME	COURSE COST	PRIMARY OBJECTIVE
ML Interns (Trial Phase)	40-50 Minutes	100% Free (Zero Paywalls)	High-Precision Classification

01 Leak-Free Pipeline Architecture with ColumnTransformer

MODULE 1

The most common flaw in production ML is preprocessing data outside of cross-validation folds. Encapsulate all feature transformations inside a Scikit-Learn **`Pipeline`** paired with a **`ColumnTransformer`**:

```
# Encapsulate transformations so fits happen strictly on training folds
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
preprocessor = ColumnTransformer([
    ('num', Pipeline([('imp', SimpleImputer(strategy='median')), ('scl', StandardScaler())]), num_cols),
    ('cat', Pipeline([('imp', SimpleImputer(strategy='constant', fill_value='UNKNOWN')), ('ohe',
OneHotEncoder(handle_unknown='ignore'))]), cat_cols)
])
full_pipeline = Pipeline([('prep', preprocessor), ('clf', RandomForestClassifier(random_state=42))])
```

02 Domain Feature Engineering for Telemetry & Fraud

MODULE 2

Raw telemetry attributes alone rarely capture subtle adversarial behavior. Feature engineering creates synthetic signals that amplify decision boundaries:

THREE CORE FEATURE ENGINEERING PATTERNS

1. Velocity & Ratio Signals: Ratios like payload-per-API-call (`request_payload_bytes / api_call_count`) expose automated bot spikes.
2. Skew Stabilization: Apply logarithmic transformations (`np.log1p`) to heavy-tailed continuous metrics like session durations to normalize gradients.
3. Out-of-Fold Target Encoding: When mapping high-cardinality country/IP features, calculate target statistics exclusively inside K-fold training splits to eliminate label leakage.

03 Tackling Severe Class Imbalance (15% Positives)

MODULE 3

When legitimate sessions vastly outnumber fraudulent sessions, models tend to predict the majority class trivially. Address this through cost-sensitive learning rather than naive oversampling:

IMBALANCE STRATEGY CHECKLIST

- ✓ **Class Weighting:** Set `class_weight='balanced'` in Random Forest or `scale_pos_weight = (n_neg / n_pos)` in XGBoost to penalize false negatives proportionally.
- ✓ **SMOTE Placement:** If using SMOTE synthetic oversampling, place it strictly inside an `imblearn.pipeline.Pipeline` so test folds remain unpolluted.
- ✓ **Threshold Calibration:** Never default to 0.50 probability threshold. Tune decision thresholds based on real operational cost trade-offs.

04 Ensemble Modeling : Random Forest vs Gradient Boosting

MODULE 4

Ensemble methods combine multiple weak learners to build robust decision boundaries. Understand the architectural differences:

PROPERTY	RANDOM FOREST (BAGGING)	XGBOOST / LIGHTGBM (BOOSTING)
Training Mechanism	Parallel, independent deep trees on bootstrap samples	Sequential shallow trees optimizing pseudo-residuals
Variance vs Bias	Primarily reduces variance of high-depth trees	Primarily reduces bias through gradient descent
Handling Outliers	Extremely resilient due to randomized feature bagging	Can overfit outliers if learning rate is excessively large
Hyperparameter Focus	`n_estimators, max_depth, min_samples_leaf`	`learning_rate, max_depth, subsample, colsample_bytree`

05 Evaluation Metrics : Why ROC-AUC is Deceptive on Imbalanced Data

MODULE 5

On datasets with rare positive classes (e.g. 15% fraud), a model can achieve a misleadingly high ROC-AUC of 0.95 while still generating hundreds of false positives. Always evaluate using Precision-Recall curves:

EVAL A Precision-Recall AUC (PR-AUC / Average Precision)

Primary Metric

Focuses exclusively on the positive minority class without rewarding the model for correctly predicting abundant benign sessions.

```
# Compute Average Precision Score on held-out test split
>>> from sklearn.metrics import average_precision_score, precision_recall_curve
>>> pr_auc = average_precision_score(y_test, y_probs[:, 1])
# Benchmark Target: PR-AUC >= 0.88 for production eligibility
```

06 Model Explainability with TreeSHAP

MODULE 6

Enterprise fraud and security pipelines cannot rely on black boxes. TreeSHAP provides game-theoretically optimal feature attribution:

```
# Compute exact TreeSHAP values on gradient boosted ensemble
>>> import shap
>>> explainer = shap.TreeExplainer(model)
>>> shap_values = explainer.shap_values(X_test_transformed)
>>> shap.summary_plot(shap_values, X_test_transformed, show=False)
```

WEEKEND 1-ON-1 MENTOR PROBE PREPARATION (GATE 2)

During your Gate 2 review call, your ML Lead will ask: (1) "Why did PR-AUC remain stable while accuracy fluctuated under different thresholds?", (2) "Prove with code that feature scaling did not leak test statistics into training folds," and (3) "Which top 3 features had the highest mean absolute SHAP value, and does their directionality make domain sense?"

07 Curated 100% Free Courses & Interactive Practice Labs

ZERO PAYWALLS

All recommended courses below are **100% free** with no paywalls or credit card requirements. They provide interactive notebooks and video walkthroughs directly aligned with Week 2 deliverables:

COURSE & PROVIDER	PLATFORM & FORMAT	DURATION	CORE SKILLS & DIRECT APPLICATION
Intro to Machine Learning Kaggle Learn	Interactive Jupyter Labs	3 Hours	Decision trees, Random Forest ensembles, underfitting vs overfitting, and model validation protocols (Task 2.1).
Intermediate Machine Learning Kaggle Learn	Interactive Coding Labs	4 Hours	Pipelines, cross-validation, XGBoost gradient boosting, and data leakage elimination (Task 2.1, 2.3).
StatQuest: Random Forests & XGBoost Josh Starmer (YouTube)	Illustrated Video Series	3 Hours	Intuitive mathematical derivations of bagging, boosting, residual optimization, and regularization parameters.
Machine Learning Crash Course Google for Education	Interactive Modules	Self-Paced	Classification metrics, Precision vs Recall tradeoffs, PR-AUC vs ROC-AUC, and regularization L1/L2 penalties.
Tabular Deep Dive fast.ai (Jeremy Howard)	Video Lectures + Labs	3-4 Hours	Feature importance analysis, out-of-fold validation, data cleaning best practices, and tree ensemble mechanics.
SHAP: Interpretable ML Official SHAP Documentation	Tutorials + Cookbooks	2 Hours	Game-theoretic Shapley values, TreeSHAP implementation, beeswarm summary plots, and local prediction waterfalls.

08 Official Documentation & Technical Cheat Sheets

PRIMARY REFERENCES

Bookmark these primary documentation hubs for immediate reference when architecting your Week 2 pipelines:

PRIMARY DOCUMENTATION INDEX

1. Scikit-Learn Pipeline & ColumnTransformer: `scikit-learn.org/stable/modules/compose.html` (Official guide on assembling composite estimators).
2. Imbalanced-Learn User Guide: `imbalanced-learn.org/stable/` (Under-sampling, over-sampling, and imbalanced pipeline chaining).
3. XGBoost Python Package Docs: `xgboost.readthedocs.io/en/stable/python/` (Parameter tuning guide for scale_pos_weight, max_depth, and subsample).
4. SHAP (SHapley Additive exPlanations): `shap.readthedocs.io/en/latest/` (TreeExplainer and model visualization API reference).
5. Joblib Persistence Guide: `joblib.readthedocs.io/en/latest/persistence.html` (Fast disk serialization for Python estimators with large NumPy arrays).

09 Recommended 5-Day Study & Execution Plan

WORKFLOW SCHEDULE

- ☐ **Day 6 (Baselines):** Review Scikit-Learn Pipeline docs & establish trivial benchmark baselines.
- ☐ **Day 7 (Pipeline):** Implement `src/pipeline.py` & run stratified 5-fold cross-validation.
- ☐ **Day 8 (Feature Eng):** Engineer ratio & log features; test class weighting vs SMOTE.
- ☐ **Day 9 (Ensembles):** Train XGBoost/Random Forest; generate SHAP summary beeswarm figure.
- ☐ **Day 10 (Review):** Author Model Card memo, serialize model artifact, and push PR by Friday 11:59 PM.
- ☐ **Weekend:** Attend Gate 2 1-on-1 code review call to unlock Week 3 Attenomy Cortex tasks.